A PERFORMANCE ON RECENTLY PUBLISHED MODELS

Tables A.1 and A.2 combine the results from the main paper with one new run of GPT-6 Luna and GPT-6.1 Sol on each of the 69 CSVStorm files. New model names are marked "(new)". Both models used the default medium reasoning effort and fresh API requests without local response-cache reuse. The Hybrid+DuckDB variant was evaluated only with the new models. Figure A.1 compares exact reconstruction accuracy for GPT-5.4 Mini, GPT-5.6 Luna, GPT-5.6 Sol, GPT-6 Luna, and GPT-6.1 Sol across the hybrid, full-agentic, and custom-parser-code approaches. API cost is shown where the runs saved token usage. GPT-5.6 Luna was not evaluated with the full-agentic approach, and the custom parser code did not record API token usage.

	System	SuT Success [%]	Pollock Score	Accuracy [%]
Hybrid	GPT-5.6 Sol	100.0	9.94	82.6
	GPT-5.6 Luna	100.0	9.71	73.9
	GPT-5.4 Mini	94.2	9.23	63.8
	Qwen3.5 0.8B	100.0	7.53	27.5
	GPT-6 Luna (new)	100.0	9.93	79.7
	GPT-6.1 Sol (new)	100.0	9.94	84.1
H+DB	GPT-6 Luna (new)	100.0	9.93	84.1
	GPT-6.1 Sol (new)	**100.0**	**9.94**	**85.5**
Full Agentic	GPT-5.6 Sol	98.6	9.57	73.9
	GPT-5.4 Mini	53.6	5.21	37.7
	GPT-6 Luna (new)	89.9	8.67	58.0
	GPT-6.1 Sol (new)	94.2	9.02	66.7
Custom Parser Code	GPT-5.6 Sol	68.1	6.71	44.9
	GPT-5.6 Luna	79.7	7.75	49.3
	GPT-5.4 Mini	95.7	8.46	34.8
	Qwen3.5 0.8B	0.0	0.00	0.0
	GPT-6 Luna (new)	95.7	9.37	63.8
	GPT-6.1 Sol (new)	97.1	9.56	62.3
Baselines	CleverCSV	100.0	8.67	55.1
	DuckDB Parse*	100.0	9.27	40.6
	DuckDB Auto	98.6	8.22	34.8
	PyCSV	94.2	8.73	10.1
	Pandas*	98.6	8.06	33.3

* denotes oracle-assisted parsing with access to file-format metadata such as the dialect.

Table A.1: CSVStorm results on 68 irregular and one control file, extending Table 1. SuT Success measures non-crashing runs. Accuracy measures exact reconstruction. H+DB adds DuckDB dialect sniffing to the hybrid parser. "(new)" identifies the new GPT-6 runs.

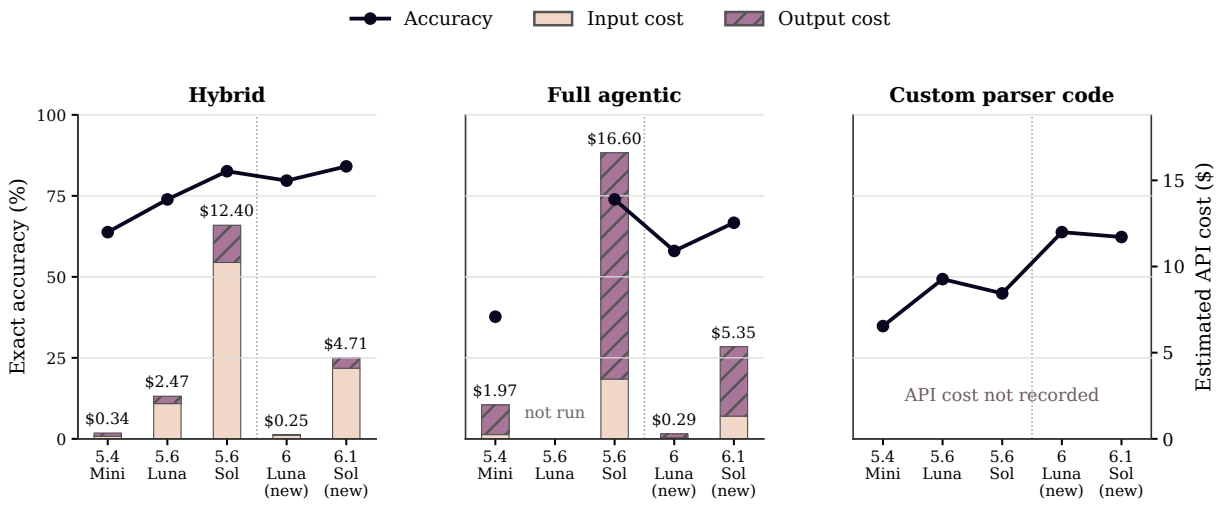


Figure A.1: Exact reconstruction accuracy (line, left axis) and estimated API cost (stacked bars, right axis) for one 69-file CSVStorm pass with the selected GPT-5.4, GPT-5.6, and GPT-6 models. Each panel is a parser method; bar segments separate input (including cached input) from output cost. Both axes use the same scale across panels. GPT-5.6 Luna has no full-agentic run; the custom parser code saved no API token usage, so its costs are unavailable. Older hybrid bar totals match Table A.2; their segment shares are estimated from recorded input and output tokens. Other costs use saved token counts and the model rates recorded in the implementation. These are estimates rather than invoice totals.

Model	Acc.	Time		Cost	
	(%)↑	Median (s/file)↓	Total (s)↓	Median ($/file)↓	Total ($)↓
GPT-5.6 Sol	82.6	5.57	1092.54	0.0177	12.3958
GPT-5.6 Luna	73.9	4.64	563.20	0.0049	2.4736
GPT-5.4 Mini	63.8	**2.34**	**269.22**	0.0024	0.3375
Qwen3.5 0.8B	27.5	12.08	1402.32	Local	Local
GPT-6 Luna (new)	79.7	5.74	886.38	**0.00046**	**0.2518**
GPT-6.1 Sol (new)	**84.1**	3.74	618.28	0.00627	4.7109

Table A.2: Performance, runtime, and cost of the hybrid repair approach, extending Table 2. "(new)" identifies the new GPT-6 runs. Local models incur no API charge. New-run costs use recorded token usage and standard API rates and are not invoice totals.

The new timing measurements include network and provider latency and were made while other benchmark configurations were running. The custom parser code implementation reads generated code from each HTTP response but does not save the response's token-usage field. Its benchmark records runtime and call counts without token counts, so exact API cost cannot be recovered from these runs.